

Clinic Appointment System

1. Introduction

The **Clinic Appointment System** is a web-based application to allow patients to book, reschedule, and cancel appointments with doctors. Use Code first Approach, Asp.net core MVC and Entity framework core to implement the application

2. System Requirements

2.1 Functional Requirements

- **User Authentication:** Patients, doctors, and administrators can register, log in, and manage their profiles.
- **Appointment Booking:** Patients can book appointments with available doctors.
- **Rescheduling & Cancellation:** Patients can reschedule or cancel their appointments.
- **Appointment Status Tracking:** Patients and doctors can view upcoming and past appointments.
- **Admin Dashboard:** Manage users, doctors, and appointment records.

3. Entity Classes & Properties

3.1 User Entity

- **Id**
- **FirstName**
- **LastName**
- **Email**
- **Password**
- **Role** // Patient, Doctor, Admin

3.2 Doctor Entity

- **Id { get; set; }**
- **Name { get; set; }**
- **Specialty { get; set; }**
- **public List<Appointment> Appointments**

3.3 Appointment Entity

- **Id**
- **PatientId**
- **User Patient**
- **DoctorId**
- **Doctor Doctor**

- **AppointmentDate**
- **Status** // Scheduled, Completed, Canceled

4. Repository Interfaces

4.1 IAppointmentRepository

- Task<IEnumerable<Appointment>> GetAllAppointmentsAsync(int userId);
- Task<Appointment> GetAppointmentByIdAsync(int id);
- Task AddAppointmentAsync(Appointment appointment);
- Task UpdateAppointmentStatusAsync(int id, string status);
- Task DeleteAppointmentAsync(int id);

4.2 IDoctorRepository

- Task<IEnumerable<Doctor>> GetAllDoctorsAsync();
- Task<Doctor> GetDoctorByIdAsync(int id);
- Task AddDoctorAsync(Doctor doctor);
- Task UpdateDoctorAsync(Doctor doctor);
- Task DeleteDoctorAsync(int id);

5. Controllers

5.1 AppointmentController

5.2 DoctorController

6. Custom Exceptions

6.1 AppointmentNotFoundException

6.2 DoctorNotFoundException